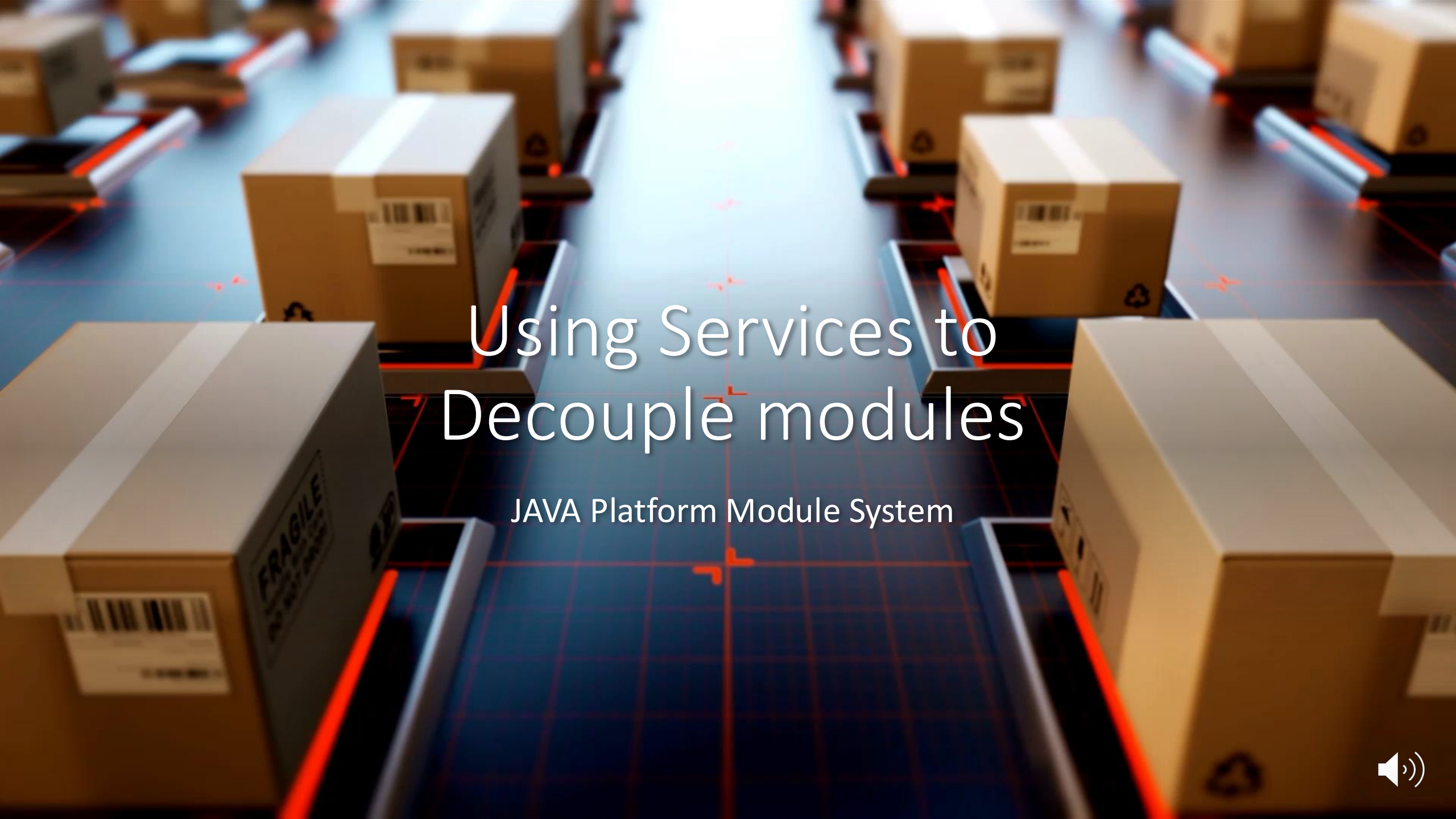



# Using Services to Decouple modules

JAVA Platform Module System

# Lecture

- Improving project designs with services
- Creating services, consumers, and providers in the JPMS
- Using the `ServiceLoader` to consume services
- Developing well-designed services
- Deploying services in plain and modular JARs across different Java versions

# Improving Design

**Listing 10.1 Depends on a concrete type and establishes the dependency**

```
public class InputStreamAwesomizer {

        private final ByteArrayInputStream stream;

        public AwesomeInputStream(byte[] buffer) {
                stream = new ByteArrayInputStream(buffer);
        }

        // [... awesome methods ...]

}
```

Depends on a concrete type

Establishes the dependency directly

**Listing 10.2 Depends on an abstract type; caller establishes the dependency**

```
public class InputStreamAwesomizer {

        private final InputStream stream;

        public AwesomeInputStream(InputStream stream) {
                this.stream = stream;
        }

        // [... awesome methods ...]

}
```

Depends on an abstract type

Dependency established by the caller

# Improving Design

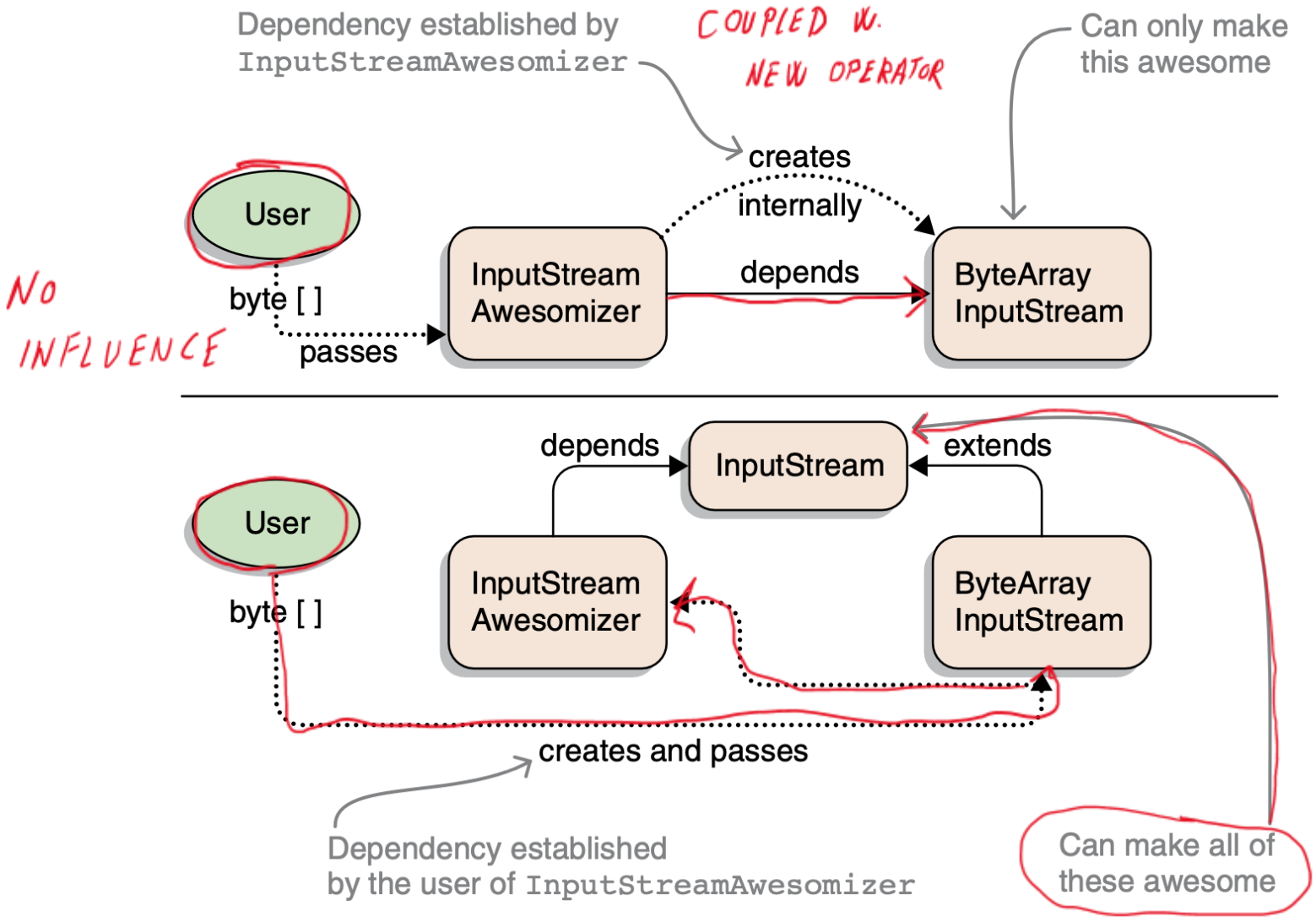



**Figure 10.1** If a type establishes its own dependencies (top), users can't influence them. If a type's dependencies are passed during construction (bottom), users can pick the implementation that best fits their use case.

# Improving Design

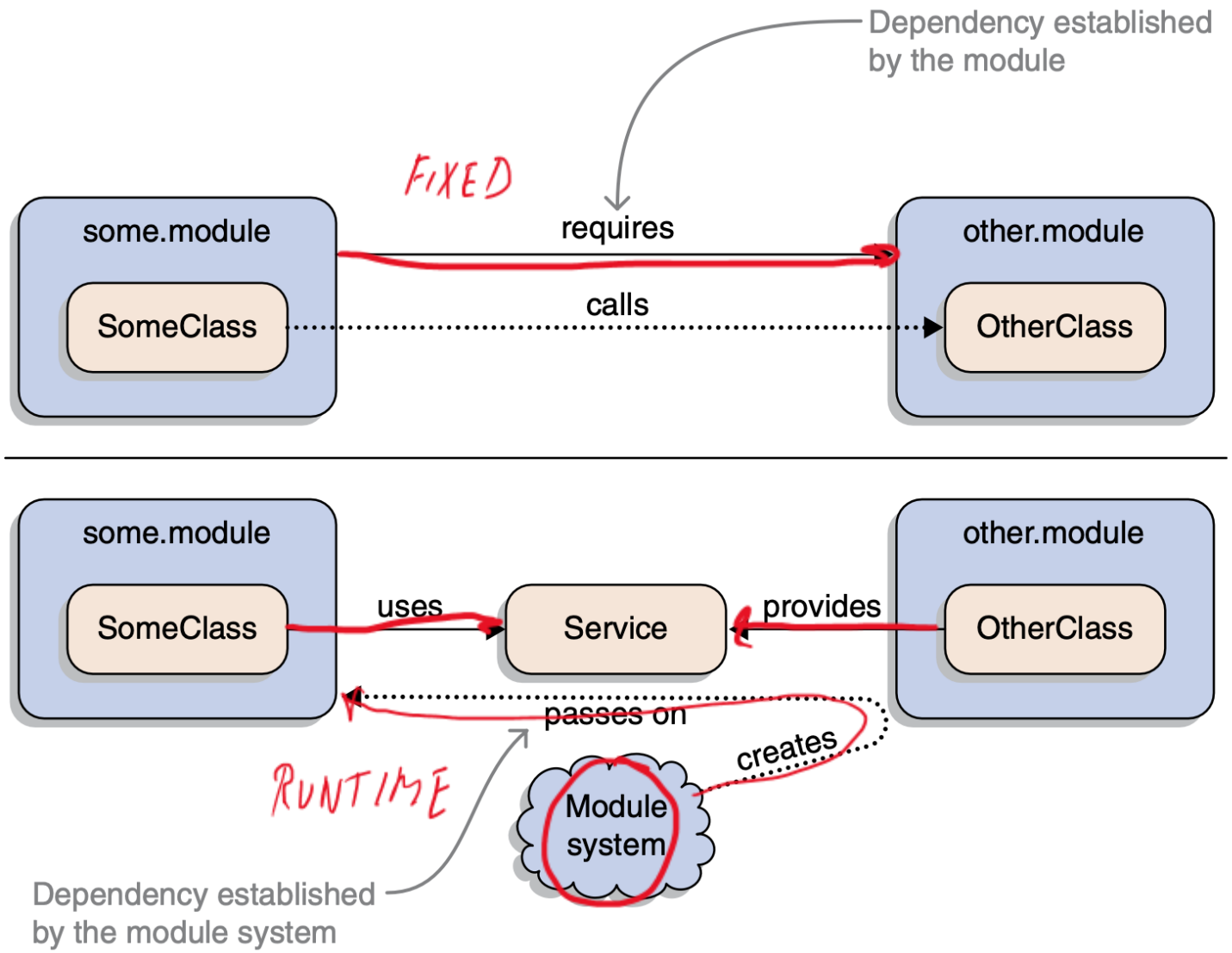



**Figure 10.2** If a module requires another (top), the dependency is fixed; it can't be changed from the outside. On the other hand, if a module uses a service (bottom), the concrete implementation is chosen at run time.

# Creating Services

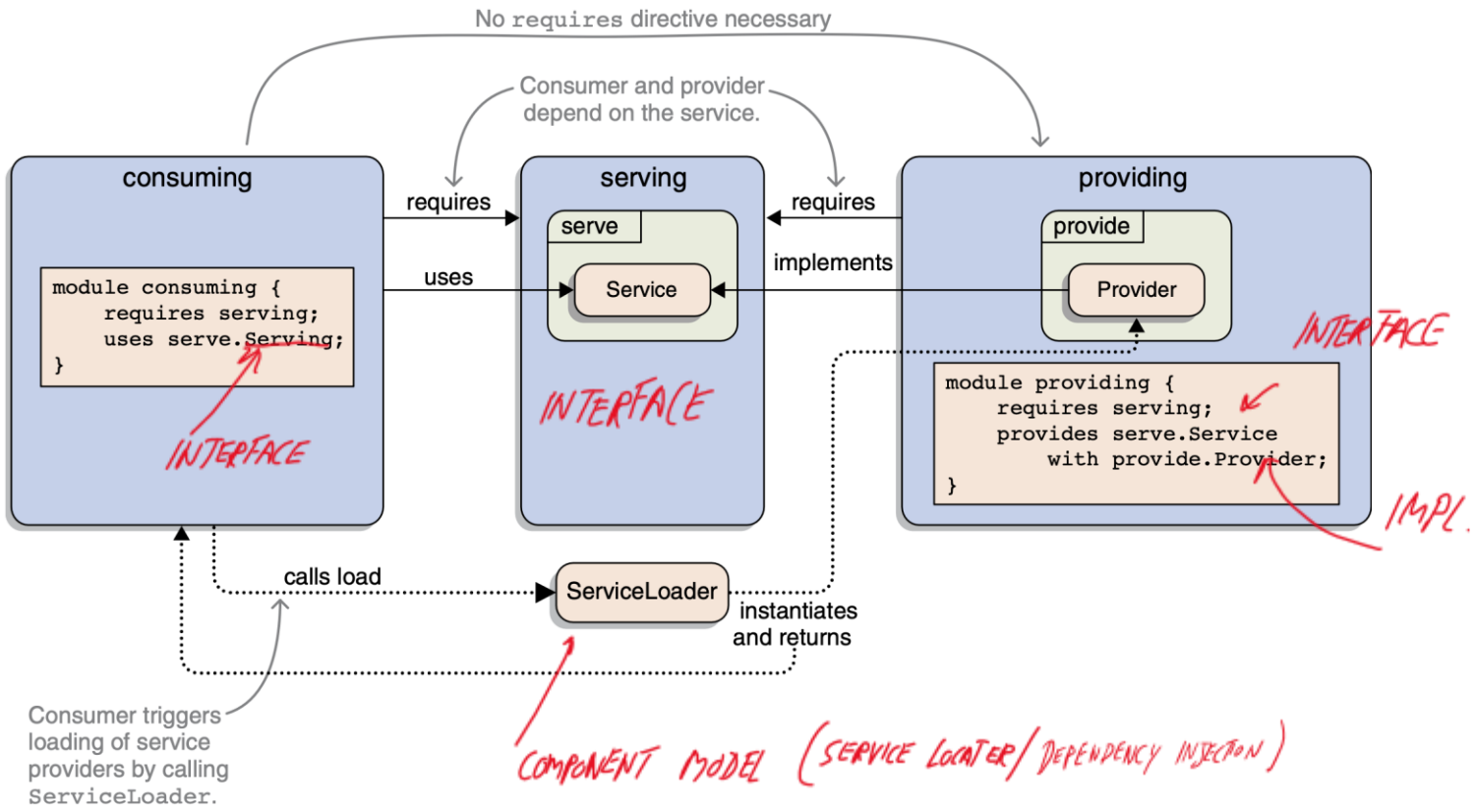



**Figure 10.3 At the center of using services is a specific type, here called `Service`. The class `Provider` implements it, and the module containing it declares that with a `provides ... with` directive. Modules consuming services need to declare that with a `uses` directive. At run time, they can then use the `ServiceLoader` to get instances of all providers for a given service.**

# Consumers

- The module *consuming* the service expresses its requirement with a `uses ${service}` *directive* in its module descriptor, where `${service}` is the fully qualified name of the service type.

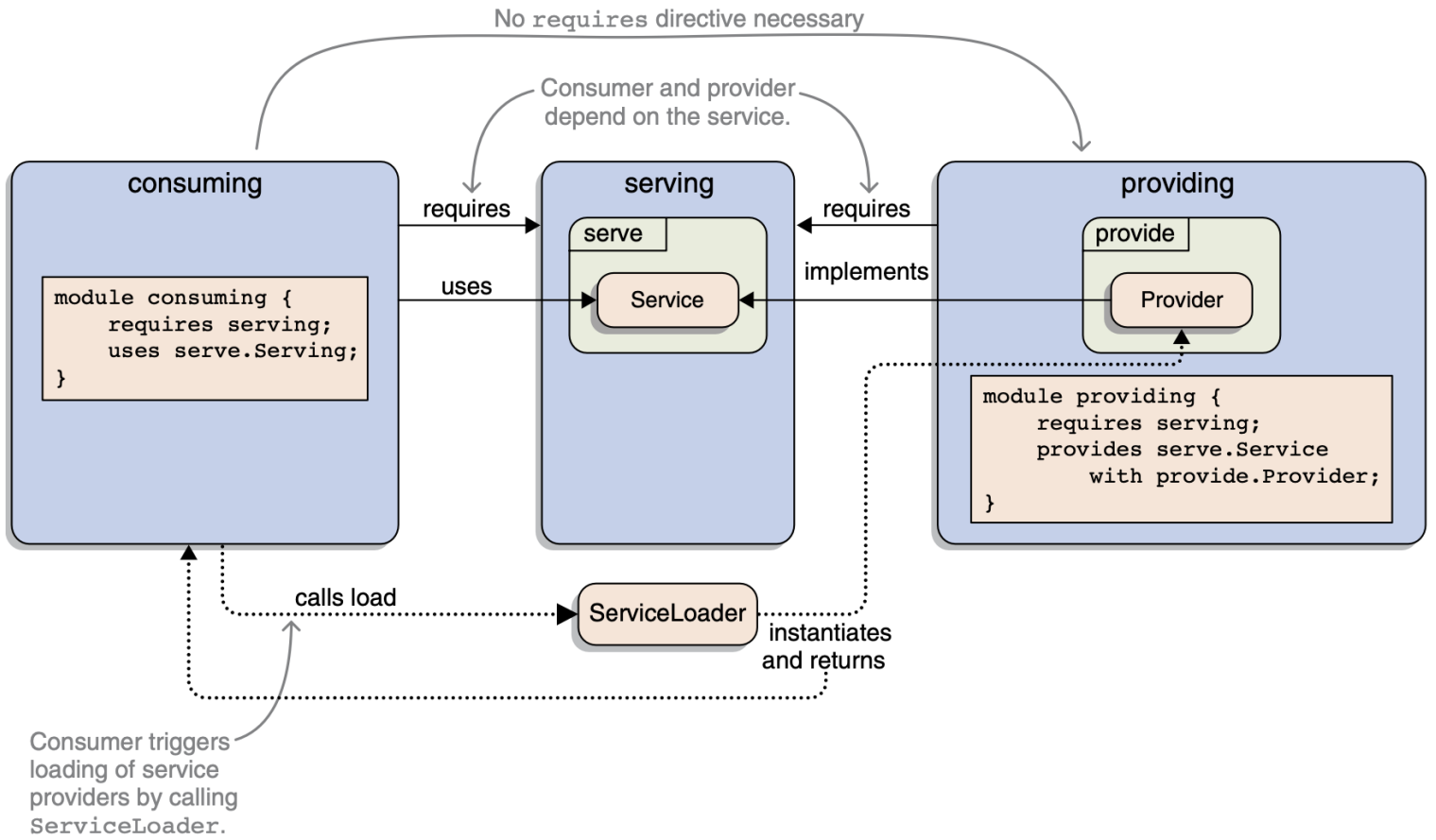
module Core { uses dk.sdu.service.IEntityProcessingService }

# Providers

- The module *providing* the service expresses its offer with a `provides ${service} with ${provider}` *directive*, where `${service}` is the same type as in the `uses` directive and `${provider}` the fully qualified name of another class, which is one or the other of the following:
  - A concrete class that extends or implements `${service}` and has a public, parameterless constructor (called a *provider constructor*)
  - An arbitrary type with a public, static, parameterless method `provide` that returns a type that extends or implements `${service}` (called a *provider method*)

module Player { provides ITentityProcessingService with dk.sdu.PlayerControlSystem }

# Example


No requires directive necessary
Consumer and provider depend on the service.
consuming
module consuming {
requires serving;
uses serve.Service;
}
requires
uses
serving
serve
Service
requires
implements
providing
provide
Provider
module providing {
requires serving;
provides serve.Service
with provide.Provider;
}
calls load
ServiceLoader
instantiates and returns
Consumer triggers loading of service providers by calling ServiceLoader.

# Example

```
module monitor {
    // [... truncated requires directives ...]
    // removed dependencies on monitor.observer.alpha and beta - yay!
    uses monitor.observer.ServiceObserver;
}
```

The second step is to declare the `provides` directives in the provider modules *monitor.observer.alpha* and *monitor.observer.beta*:

```
module monitor.observer.alpha {
    requires monitor.observer;
    // removed export of monitor.observer.alpha - yay!
    provides monitor.observer.ServiceObserver
        with monitor.observer.alpha.AlphaServiceObserver;
}
```

This doesn't work, though—the compiler throws an error:

```
> The service implementation does not have
> a public default constructor:
>     AlphaServiceObserver
```

So, create a new interface, `ServiceObserverFactory`, in *monitor.observer*. It has a single method, `createIfMatchingService`, that expects the service URL and returns an `Optional<ServiceObserver>`. In *monitor.observer.alpha* and *monitor.observer.beta*, create implementations that do what the static factory methods on `AlphaServiceObserver`

# Example

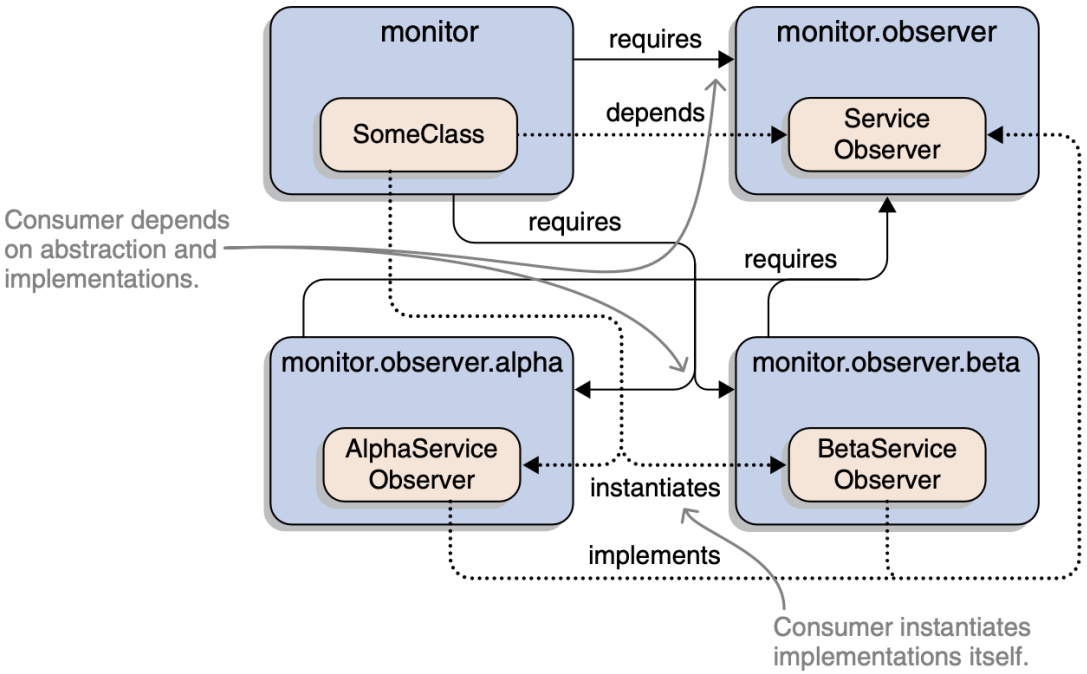



Figure 10.4 Without services, the monitor module needs to depend on all other involved modules: *observer*, *alpha*, and *beta*, as shown in this partial module graph.

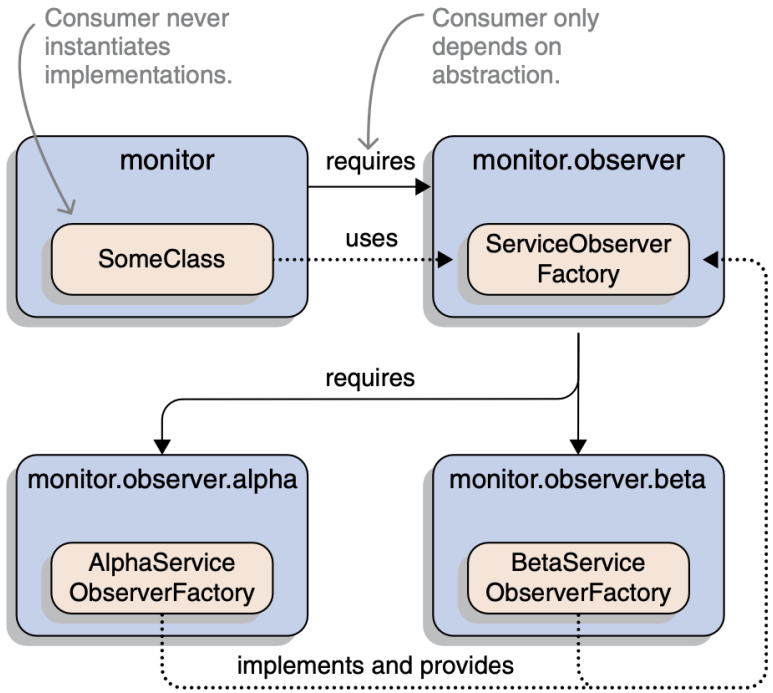



Figure 10.5 With services, *monitor* only depends on the module defining the service: *observer*. The providing modules, *alpha* and *beta*, are no longer directly required.

# Example

The consuming module monitor requires monitor.observer because it contains ServiceObserverFactory. Thanks to services, it requires neither alpha nor beta.

Nothing changed for monitor.observer: it's unaware that it's used as a service. All that's needed is the usual export of the package containing ServiceObserver and ServiceObserverFactory.

```
module monitor {
    requires monitor.observer;
    // [... truncated other requires directives ...]
    uses monitor.observer.ServiceObserverFactory;
}

module monitor.observer {
    exports monitor.observer;
}
```

The consuming module monitor uses the service interface ServiceObserverFactory.

Both provider modules require monitor.observer because they implement the interfaces it contains—services changed nothing.

```
module monitor.observer.alpha {
    requires monitor.observer;
    provides monitor.observer.ServiceObserverFactory
        with monitor.observer.alpha.AlphaServiceObserverFactory;
}

module monitor.observer.beta {
    requires monitor.observer;
    provides monitor.observer.ServiceObserverFactory
        with monitor.observer.beta.BetaServiceObserverFactory;
}
```

Each provider module provides the service ServiceObserverFactory with its concrete class.

# Consumers via ServiceLoader

The final step is to get the observer factories in *monitor*. To that end, call `ServiceLoader.load(ServiceObserverFactory.class)`, stream over the returned providers, and get the service implementations:

```
List<ServiceObserverFactory> observerFactories = ServiceLoader
    .load(ServiceObserverFactory.class).stream()
    .map(Provider::get)
    .collect(toList());
```

**Provider::get instantiates a provider (see section 10.4.2).**

# ServiceLoader

**Table 10.1** `ServiceLoader` **API at a glance**

| Return type | Method name | Description |
|---|---|---|
| **Methods to create a new service loader for the given type** | | |
| `ServiceLoader<S>` | `load(Class<S>)` | Loads providers starting from the current thread's context class loader |
| `ServiceLoader<S>` | `load(Class<S>, ClassLoader)` | Loads providers starting from the specified class loader |
| `ServiceLoader<S>` | `load(ModuleLayer, Class<S>)` | Loads providers starting from modules in the given module layer |
| `ServiceLoader<S>` | `loadInstalled(Class<S>)` | Loads providers from the platform class loader |
| **Methods to access service providers** | | |
| `Optional<S>` | `findFirst()` | Loads the first available provider |
| `Iterator<S>` | `iterator()` | Returns an iterator to lazily load and instantiate available providers |
| `Stream<Provider<S>>` | `stream()` | Returns a stream to lazily load available providers |
| `void` | `reload()` | Clears this loader's provider cache so all providers will be reloaded |

# ServiceLoader

Create ServiceLoader:

```
ServiceLoader<TheService> loader = ServiceLoader.load(TheService.class);
```

Accessing services

With a `ServiceLoader` instance for the desired service in hand, it's time to start using those providers. There are two and a half methods for doing that:

- `Iterator<S> iterator()` lets you iterate over the instantiated service providers.
- `Optional<S> findFirst()` uses `iterator` to *return the first provider* if any were found (this is a convenience method, so I only count it as a half).
- `Stream<Provider<S>> stream()` lets you *stream over service providers*, which are wrapped into a `Provider` instance. (What's up with that? Section 10.4.2 explains.)

If you have specific laziness/caching needs (see section 10.4.2 for more), you may want to keep the `ServiceLoader` instance around. But in most cases that isn't necessary and you can immediately start iterating over or streaming the providers:

```
ServiceLoader
    .load(TheService.class)
    .iterator()
    .forEachRemaining(TheService::doTheServiceThing);
```

# Summary

- The service architecture is made up of four parts:
  - The *service* is a class or an interface.
  - The *provider* is a concrete implementation of the service.
  - The *consumer* is any piece of code that wants to use a service.
  - The `ServiceLoader` creates and returns an instance of each provider of a given service to consumers.
- Requirements and recommendations for the service type are as follows:
  - Any class or interface can be a service, but because the goal is to provide maximum flexibility to consumers and providers, it's recommended to use interfaces (or, at the least, abstract classes).
  - Service types need to be public and in an exported package. This makes them part of their module's public API, and they should be designed and maintained appropriately.
  - The declaration of the module defining a service contains no entry to mark a type as a service. A type becomes a service by consumers and providers using it as one.
  - Services rarely emerge randomly, but are specifically designed for their purpose. Always consider making the used type not the service but a factory for it. This makes it easier to search for a suitable implementation as well as to control when instances are created and in which state.

# Summary (Providers)

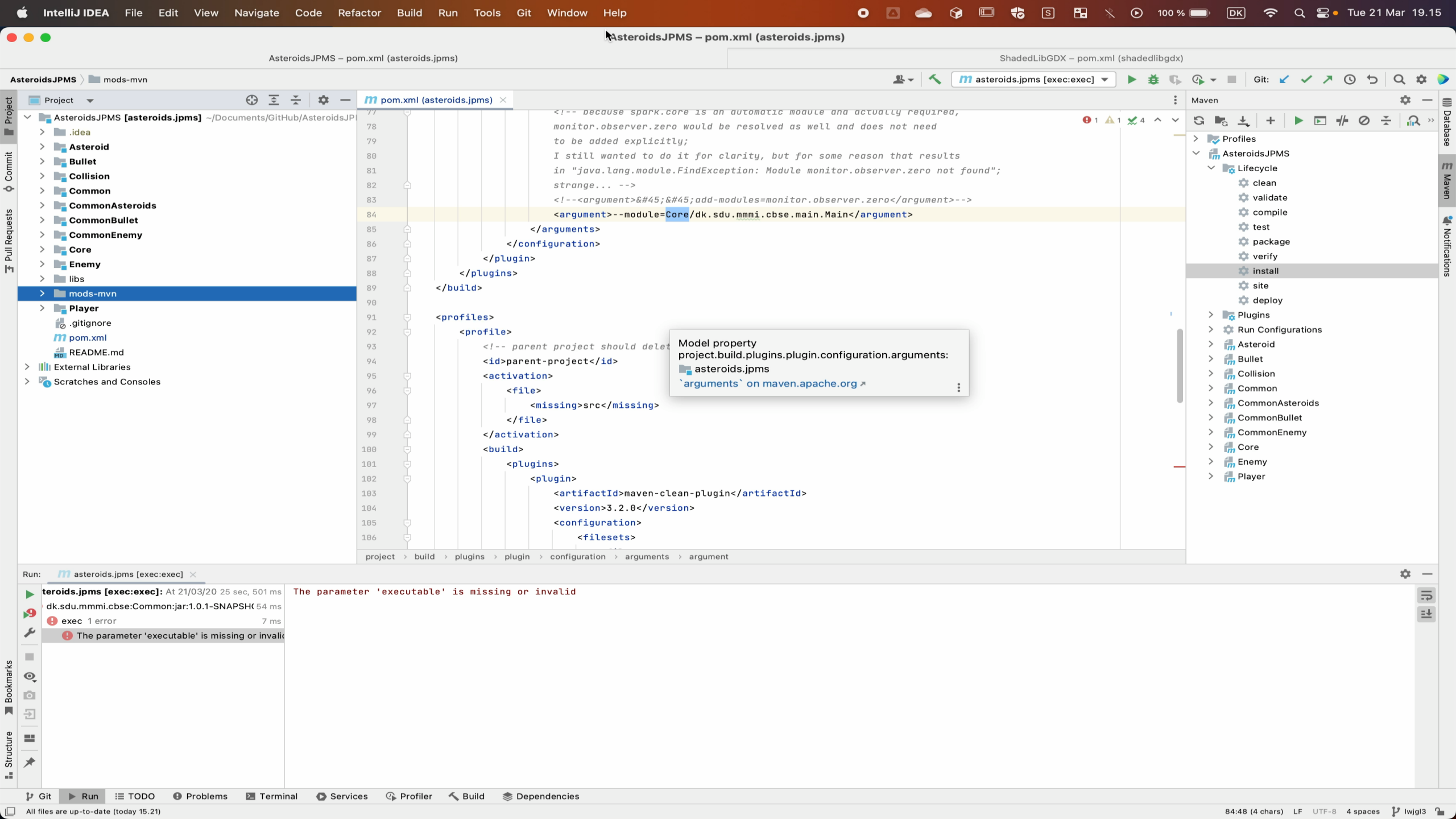
- Requirements and recommendations for providers are as follows:
  - Modules providing services need to access the service type, so they must require the module containing it.
  - There are two ways to create a service provider: a concrete class that implements the service type and has a provider constructor (a public, parameterless constructor), or a type with a provider method (a public, static, parameterless method called `provide`) that returns an instance implementing the service type. Either way, the type must be public, but there's no need to export the package containing it. On the contrary, it's advisable not to make the providing type part of a module's public API.
  - Modules providing services declare that by adding a `provides ${service} with ${provider}` directive to their descriptor.
  - If a modular JAR is supposed to provide services even if placed on the class path, it also needs entries in the `META-INF/services` directory. For each `provides ${service} with ${provider}` directive, create a plain file called `${service}` that contains one line per `${provider}` (all names must be fully qualified).

# Summary (Consumers)

- Requirements and recommendations for consumers are as follows:
  - Modules consuming services need to access the service type, so they must require the module containing it. They shouldn't require the modules providing that service, though—on the contrary, that would be against the main reason to use services in the first place: to decouple consumers and providers.
  - There's nothing wrong with service types and the service's consumers living in the same module.
  - Any code can consume services regardless of its own accessibility, but the module containing it needs to declare which services it uses with a `uses` directive. This allows the module system to perform service binding efficiently and makes module declarations more explicit and readable.
  - Modules are consumed by calling `ServiceLoader::load` and then iterating or streaming over the returned instances by calling either `iterate` or `stream`. It's possible that will be providers are found, and consumers must handle that case gracefully.

# Summary (Consumers)

- The behavior of code that consumes services depends on global state: which provider modules are present in the module graph. This gives such code undesirable properties like making it hard to test. Try to push service loading into setup code that creates objects in their correct configuration (for example, your dependency injection framework), and always allow regular provider code to pass service providers to consuming classes (for example, during construction).
- The service loader instantiates providers as late as possible. Its `stream` method even returns a `Stream<Provider<S>>`, where `Provider::type` can be used to access the `Class` instance for the provider. This allows searching for a suitable provider by checking class-level annotations without instantiating the provider yet.
- Service-loader instances aren't thread-safe. If you use them concurrently, you have to provide synchronization.
- All problems during loading and instantiating providers are thrown as `ServiceConfigurationError`. Due to the loader's laziness, this doesn't happen during `load`, but later in `iterate` or `stream` when problematic providers are encountered. Always be sure to put the entire interaction with `ServiceLoader` into a `try` block if you want to handle errors.


IntelliJ IDEA File Edit View Navigate Code Refactor Build Run Tools Git Window Help
Tue 21 Mar 19.15
AsteroidsJPMS – pom.xml (asteroids.jpms)
AsteroidsJPMS – pom.xml (asteroids.jpms)
ShadedLibGDX – pom.xml (shadedlibgdx)
AsteroidsJPMS
mods-mvn
asteroids.jpms [exec:exec]
Git:
Project
AsteroidsJPMS [asteroids.jpms] ~/Documents/GitHub/AsteroidsJP
.idea
Asteroid
Bullet
Collision
Common
CommonAsteroids
CommonBullet
CommonEnemy
Core
Enemy
libs
mods-mvn
Player
.gitignore
pom.xml
README.md
External Libraries
Scratches and Consoles
pom.xml (asteroids.jpms)
<!-- because spark.core is an automatic module and actually required,
monitor.observer.zero would be resolved as well and does not need
to be added explicitly;
I still wanted to do it for clarity, but for some reason that results
in "java.lang.module.FindException: Module monitor.observer.zero not found";
strange... -->
<!--<argument>--add-modules=monitor.observer.zero</argument>-->
<argument>--module=Core/dk.sdu.mmmi.cbse.main.Main</argument>
</arguments>
</configuration>
</plugin>
</plugins>
</build>
<profiles>
<profile>
<!-- parent project should delet
<id>parent-project</id>
<activation>
<file>
<missing>src</missing>
</file>
</activation>
<build>
<plugins>
<plugin>
<artifactId>maven-clean-plugin</artifactId>
<version>3.2.0</version>
<configuration>
<filesets>
Model property project.build.plugins.plugin.configuration.arguments:
asteroids.jpms
`arguments` on maven.apache.org
project › build › plugins › plugin › configuration › arguments › argument
Maven
Profiles
AsteroidsJPMS
Lifecycle
clean
validate
compile
test
package
verify
install
site
deploy
Plugins
Run Configurations
Asteroid
Bullet
Collision
Common
CommonAsteroids
CommonBullet
CommonEnemy
Core
Enemy
Player
Run: asteroids.jpms [exec:exec]
teroids.jpms [exec:exec]: At 21/03/20 25 sec, 501 ms
dk.sdu.mmmi.cbse:Common:jar:1.0.1-SNAPSH 54 ms
exec 1 error 7 ms
The parameter 'executable' is missing or invali
The parameter 'executable' is missing or invalid
Git
Run
TODO
Problems
Terminal
Services
Profiler
Build
Dependencies
All files are up-to-date (today 15.21)
84:48 (4 chars)
LF
UTF-8
4 spaces
lwjgl3